

Empire AI Beta NVL72 Cluster - Job Submission Guide

72 nodes | 288 NVIDIA GB200 GPUs | QoS tiers, setup commands, and reference

Setup, Test & Verify

Section 1 - GPU Access, Containers & Storage

Beta has no pre-installed Python or PyTorch - everything runs inside NGC containers via Pyxis/Enroot. Run these three commands in order to verify your environment is working.

Test 1: Verify Slurm scheduling + GPU driver - confirms your account can schedule a GPU job and the driver responds correctly. This is a quick connectivity check (no container needed).

```
srun --partition=beta --account=YOUR_ACCOUNT --qos=test \
--gres=gpu:1 --time=00:05:00 \
bash -c "nvidia-smi --query-gpu=name,driver_version,memory.total --format=csv"
```

Expected: One row showing NVIDIA GB200, driver version 580.x, and ~189 GB memory.

Test 2: Container pull, GPU passthrough & multi-GPU [CONTAINER TEST] - pulls the NGC PyTorch image via Pyxis/Enroot (~15 GB, first run takes a few minutes), injects GPUs into the container, and confirms all 4 GPUs are visible inside PyTorch.

```
srun --partition=beta --account=YOUR_ACCOUNT --qos=test \
--gres=gpu:4 --time=00:10:00 \
--container-image=nvcr.io/nvidia/pytorch:26.02-py3 \
bash -lc "python -c 'import torch; print(f\"GPUS: {torch.cuda.device_count()}\"); print(torch.cuda.get_device_name(0))'"
```

Expected: GPUS: 4 and NVIDIA GB200. First run downloads the ~15 GB image; subsequent runs with the same image are faster.

Test 3: Home directory mount [CONTAINER TEST] - confirms your home directory (/home/YOUR_USER) on shared VAST storage is accessible inside the container via --container-mount-home.

```
srun --partition=beta --account=YOUR_ACCOUNT --qos=test \
--gres=gpu:1 --time=00:05:00 \
--container-image=nvcr.io/nvidia/pytorch:26.02-py3 \
--container-mount-home \
bash -c "ls ~/ | head -5"
```

Expected: You should see your home directory files listed. Use --container-mount-home in your jobs whenever you need to read input data or write results to your home directory.

Section 2 - Key Flags

Flag	Purpose	Example
--partition=	Beta partition (required)	--partition=beta
--qos=	QoS tier	--qos=standard
--gres=gpu:N	Number of GPUs per node (max 4)	--gres=gpu:4 (full node)
--time=	Max wall time	--time=24:00:00 or --time=2-00:00:00

Flag	Purpose	Example
--nodes=	Multi-node (each node = 4 GPUs)	--nodes=4 --gres=gpu:4 = 16 GPUs
--requeue	Auto-restart when wall time hits	--requeue --signal=B:SIGTERM@900
--array=	Run N independent tasks, each gets \$SLURM_ARRAY_TASK_ID	--array=1-100
--account=	Project account (required)	--account=YOUR_ACCOUNT
--container-image=	Run inside an NGC container	--container-image=nvcr.io/nvidia/pytorch:26.02-py3
--container-name=	Cache container locally (skip re-download on next run). Note: do not combine with <code>bash -lc torch</code> commands - use <code>--container-image</code> only for PyTorch jobs.	--container-name=pytorch_test
--container-mount-home	Mount your home directory inside the container. Note: if your home has <code>conda/pip</code> , this can override the container's Python environment - omit for <code>srun torch</code> commands; safe with <code>sbatch</code> scripts.	--container-mount-home

GPU Hardware

GPU	VRAM	Nodes	GPUs/Node
NVIDIA GB200 (NVL72)	189 GB HBM3e	72 nodes (288 GPUs total)	4

Section 3 - QoS Tier Smoke Tests

Submit one job per tier to confirm they work. All should complete successfully.

Test 4: test - half price, max 6h, 4 GPUs/job

```
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=test --gres=gpu:1 --time=00:05:00 --wrap="echo
'test tier works on' \$(hostname); nvidia-smi; sleep 10"
```

Test 5: standard - production default, max 48h, 36 GPUs/job

```
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=standard --gres=gpu:1 --time=00:05:00 --wrap="echo
'standard tier works on' \$(hostname); nvidia-smi; sleep 10"
```

Test 6: long - half price, lower priority, max 7d, 36 GPUs

```
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=long --gres=gpu:1 --time=00:05:00 --wrap="echo
'long tier works on' \$(hostname); nvidia-smi; sleep 10"
```

Test 7: priority - 2x cost, highest priority, max 24h, 72 GPUs

```
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=priority --gres=gpu:1 --time=00:05:00 --wrap="echo
'priority tier works on' \$(hostname); nvidia-smi; sleep 10"
```

Test 8: interactive - live GPU shell, max 2h, 4 GPUs, 1 session

```
srun --partition=beta --account=YOUR_ACCOUNT --qos=interactive --gres=gpu:1 --time=00:10:00 --pty bash
```

Once inside, run:

```
nvidia-smi
```

Then exit the interactive session (**important - do this before running Test 9**):

```
exit
```

Test 9: Containerized QoS smoke test [CONTAINER TEST] - confirms a real PyTorch workload runs inside an NGC container with GPU access.

```
srunc --partition=beta --account=YOUR_ACCOUNT --qos=standard \
--gres=gpu:1 --time=00:05:00 \
--container-image=nvcr.io/nvidia/pytorch:26.02-py3 \
bash -lc "python -c 'import torch; print(f\"GPU: {torch.cuda.get_device_name(0)}\"); print(f\"CUDA: {
torch.version.cuda}\")'"
```

Expected: Job completes showing NVIDIA GB200 and CUDA version. All production jobs on Beta should use NGC containers via Pyxis/Enroot - there is no pre-installed Python or PyTorch on compute nodes.

Section 4 - Job Script & Training Template

Copy and paste to create these files in your home directory.

train.sh - Slurm job script with requeue + checkpoint signal

```
[ -f ~/train.sh ] && cp ~/train.sh ~/train.sh.bak.$(date +%Y%m%d_%H%M%S) && echo "Backed up ~/train.sh"
cat > ~/train.sh << 'EOF'
#!/bin/bash
#SBATCH --job-name=my_training
#SBATCH --partition=beta
#SBATCH --account=YOUR_ACCOUNT
#SBATCH --qos=standard
#SBATCH --gres=gpu:4
#SBATCH --time=12:00:00
#SBATCH --requeue
#SBATCH --signal=B:SIGTERM@900
#SBATCH --output=%x_%j.out
#SBATCH --error=%x_%j.err
#SBATCH --container-image=nvcr.io/nvidia/pytorch:26.02-py3

python train.py
EOF
chmod +x ~/train.sh && echo "Created ~/train.sh"
```

train.py - PyTorch checkpoint template. Saves on SIGTERM, resumes from latest checkpoint, keeps last 3.

```
[ -f ~/train.py ] && cp ~/train.py ~/train.py.bak.$(date +%Y%m%d_%H%M%S) && echo "Backed up ~/train.py"
cat > ~/train.py << 'PYEOF'
import signal, sys, os, glob, torch

CKPT_DIR = os.path.join(os.environ["HOME"], "checkpoints")
MAX_CKPTS = 3
os.makedirs(CKPT_DIR, exist_ok=True)

def save_checkpoint(model, opt, epoch):
    path = os.path.join(CKPT_DIR, f"ckpt_epoch_{epoch:04d}.pt")
    torch.save({"model": model.state_dict(),
               "optimizer": opt.state_dict(),
               "epoch": epoch}, path)
    for old in sorted(glob.glob(os.path.join(CKPT_DIR, "ckpt_epoch_*.pt"))[:-MAX_CKPTS]):
        os.remove(old)

# --- Replace these with your actual model + training loop ---
# model = YourModel()
# opt = torch.optim.Adam(model.parameters())
# num_epochs = 100

# --- Register SIGTERM handler AFTER defining model + optimizer ---
def save_and_exit(signum, frame):
    save_checkpoint(model, opt, epoch)
    sys.exit(0)

signal.signal(signal.SIGTERM, save_and_exit)

start_epoch = 0
ckpts = sorted(glob.glob(os.path.join(CKPT_DIR, "ckpt_epoch_*.pt")))
if ckpts:
    ckpt = torch.load(ckpts[-1], weights_only=False)
    model.load_state_dict(ckpt["model"])
    opt.load_state_dict(ckpt["optimizer"])
    start_epoch = ckpt["epoch"] + 1

for epoch in range(start_epoch, num_epochs):
    train(model)
    save_checkpoint(model, opt, epoch)
PYEOF
echo "Created ~/train.py"
```

Flow: sbatch ~/train.sh -> trains -> saves each epoch -> SIGTERM 15 min before wall time -> save & exit -> --requeue restarts -> resumes from checkpoint.

Section 5 - Test 10: Requeue, Checkpointing & SIGTERM

Tests the requeue + checkpoint workflow.

```
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=test --gres=gpu:1 --time=00:02:00 --requeue --
signal=B:SIGTERM@60 --wrap='trap "echo SIGTERM caught at $(date) - saving checkpoint; exit 0" TERM; echo
"Job started at $(date)"; for i in $(seq 1 120); do sleep 1; done'
```

Expected: Output shows "Job started" then "SIGTERM caught" ~1 minute into the job (60s before the 2-min wall time). Check the output (~2 min):

```
cat $(ls -t slurm-*.out | head -1)
```

Section 6 - Test 11: Checkpoint Save & Resume

Verifies checkpoint save and resume across two jobs.

Run the first job (creates a checkpoint):

```
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=test --gres=gpu:1 --time=00:05:00 --wrap='mkdir -p
~/checkpoints; echo "epoch=5 loss=0.42" > ~/checkpoints/ckpt_epoch_0005.pt; echo "Checkpoint saved: $(
cat ~/checkpoints/ckpt_epoch_0005.pt)'"
```

Wait for it to complete (re-run until no jobs appear):

```
squeue -u $USER
```

Once the queue is empty, run the resume job:

```
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=test --gres=gpu:1 --time=00:05:00 --wrap='CKPT=$(ls
-t ~/checkpoints/ckpt_epoch_*.pt 2>/dev/null | head -1); if [ -n "$CKPT" ]; then echo "Resumed from:
$CKPT -- $(cat $CKPT)"; else echo "No checkpoint found -- starting fresh"; fi'
```

Check both outputs:

```
cat $(ls -t slurm-*.out | head -1)
```

Expected: First job shows "Checkpoint saved: epoch=5 loss=0.42". Second job shows "Resumed from" with the same values.

Section 7 - Test 12: Array Jobs

Spawns 3 independent tasks, each with its own \$SLURM_ARRAY_TASK_ID.

```
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=test --gres=gpu:1 --time=00:05:00 --array=0-2 --
wrap='echo "Array task $SLURM_ARRAY_TASK_ID of job $SLURM_ARRAY_JOB_ID running on $(hostname)"; sleep 5'
```

Wait ~30 seconds for all 3 tasks to finish, then check the outputs:

```
squeue -u $USER      # re-run until empty, then:
```

```
ls slurm-*_*.out | while read f; do echo "=== $f ==="; cat "$f"; done
```

Expected: 3 output files (slurm-JOBID_0.out, _1.out, _2.out), each showing a different task ID.

Section 8 - Check Your Results

Use these commands at any time to inspect jobs, history, and account info.

```
# Running/pending jobs
squeue -u $USER

# Job history (today)
sacct -u $USER --format=JobID%10,JobName%20,QoS%12,AllocTRES%35,Elapsed,State -S today

# Output from a specific job (replace JOBID)
cat slurm-JOBID.out

# Your QoS tiers
sacctmgr show assoc where user=$USER format=Account%20,QoS%60

# SU usage
sshare -u $USER -l

# Cancel a job
scancel <jobid>
```

WARNING: Every Tier Has a Wall Time Limit - Plan for Requeue + Checkpointing

No tier allows unlimited runtime. If your training run exceeds the wall limit, **Slurm kills the job**. Most real workloads (LLM training, foundation models, multi-day experiments) will exceed these limits.

Solution (NVIDIA recommended best practice): Add `--requeue --signal=B:SIGTERM@900` to your job and implement checkpointing in your training script. Slurm sends a warning signal **15 minutes before the wall limit**, giving your script time to save a checkpoint and exit cleanly. If the job is requeued, it can resume from the last checkpoint instead of starting from scratch. See the **Setup, Test & Verify** tab for the full PyTorch code with rolling checkpoints and auto-cleanup.

WARNING: Requeue != instant restart. When your job requeues, it re-enters the pending queue and competes via **fairshare**. If others have higher priority, your job waits. See the **QoS Policy** tab for the full explanation of fairshare, preemption, and requeue mechanics.

QoS Tiers

Each job runs under a QoS tier. Click any card below to learn more.

priority 2.0x 24h 72 GPUs	test 0.5x 6h 4 GPUs	interactive 1.0x 2h 4 GPUs	standard 1.0x 48h 36 GPUs	long 0.5x 7d 36 GPUs	burst FREE 7d 36 GPUs
---	---	--	---	--	---

priority Urgent / Deadline - 2.0x SU, 24h, 72 GPUs

Highest scheduling priority. Your job runs **before** standard, long, and burst jobs. Costs **double** - use only for time-critical work.

- **Max wall time:** 24 hours
- **Max GPUs per job:** 72 GPUs (18 nodes x 4 GPUs)
- **SU cost:** 2.0x - a 4-GPU, 12h job costs **96 SU** instead of 48
- **Use for:** Conference deadlines, demo prep, urgent results
- **WARNING: 24h limit:** If your job needs more than 24 hours, add `--requeue --signal=B:SIGTERM@900` to your sbatch command and add the checkpoint code from the **Setup, Test & Verify -> Checkpointing** section to your training script

```
# Priority: 8 GPUs (2 nodes), 12 hours - runs before everything else
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=priority --requeue --signal=B:SIGTERM@900 --nodes=2
--gres=gpu:4 --time=12:00:00 train.sh
```

test Debug & Validate - 0.5x SU, 6h, 4 GPUs/job

Half-price tier for verifying your script works before committing to a full run. Limited to 4 GPUs per job (1 node).

- **Max wall time:** 6 hours
- **Max GPUs per job:** 4 GPUs (1 node)
- **SU cost:** 0.5x - a 4-GPU, 2h test costs only **4 SU**
- **Use for:** Checking scripts run without errors, testing data loading, verifying multi-GPU code
- **Not for:** Production training, anything over 6 hours, multi-node jobs

```
# Test: 1 GPU, 1 hour - half SU cost
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=test --gres=gpu:1 --time=01:00:00 my_job.sh
```

interactive Live GPU Shell - 1.0x, 2h, 4 GPUs, 1 session

Get a terminal directly on a compute node with GPUs. **One session at a time** per user.

- **Max wall time:** 2 hours
- **Max GPUs per job:** 4 GPUs (1 node)
- **Max running jobs:** 1

- **SU cost:** 1.0x standard rate
- **Use for:** Debugging with gdb/pdb, data exploration, environment setup, quick prototyping
- **Not for:** Long training runs, unattended jobs (use batch instead)

```
# Interactive shell with 1 GPU on Beta
srun --partition=beta --account=YOUR_ACCOUNT --qos=interactive --gres=gpu:1 --time=01:00:00 --pty bash
```

standard Production Training (default) - 1.0x, 48h, 36 GPUs/job

The **default tier** for real work. Any job submitted without `--qos=` will automatically run on the standard tier at the 1.0x rate.

- **Max wall time:** 48 hours (2 days)
- **Max GPUs per job:** 36 GPUs (9 nodes x 4 GPUs)
- **SU cost:** 1.0x - what you use is what you pay
- **Use for:** Model training, inference benchmarks, hyperparameter sweeps
- **Not for:** Runs over 48h (use long), urgent work (use priority)
- **WARNING: 48h limit:** Most training runs exceed 2 days. Add `--requeue --signal=B:SIGTERM@900` to your sbatch command and add the checkpoint code from the **Setup, Test & Verify -> Checkpointing** section to your training script

```
# Single node, 4 GPUs, 24 hours (with requeue + checkpoint signal)
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=standard --requeue --signal=B:SIGTERM@900 \
--gres=gpu:4 --time=24:00:00 train.sh

# Multi-node, 16 GPUs across 4 nodes, 2 days
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=standard --requeue --signal=B:SIGTERM@900 \
--gres=gpu:4 --nodes=4 --time=2-00:00:00 train.sh
```

long Budget-Friendly - 0.5x, 7 days, 36 GPUs

Half the cost of standard, but lower scheduling priority. Jobs may wait longer to start. Great for work that isn't time-sensitive.

- **Max wall time:** 7 days
- **Max GPUs per job:** 36 GPUs (9 nodes x 4 GPUs)
- **SU cost:** 0.5x - a 4-GPU, 24h job costs **48 SU** instead of 96
- **Use for:** Long training runs, overnight/weekend jobs, cost-sensitive workloads
- **Not for:** Time-sensitive work that needs to start quickly
- **WARNING: 7-day limit:** Even at 7 days, long simulations may need multiple passes. Add `--requeue --signal=B:SIGTERM@900` to your sbatch command and add the checkpoint code from the **Setup, Test & Verify -> Checkpointing** section to your training script

```
# Long run, 4 GPUs, 5 days, half SU cost (with requeue)
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=long --requeue --signal=B:SIGTERM@900 \
--gres=gpu:4 --time=5-00:00:00 train.sh
```

burst Opportunistic / System-Assigned Only - FREE, 7 days, 36 GPUs

You don't choose this tier - the system assigns it automatically via `budget_enforcer.sh` when your project's SU budget is exhausted. Jobs are free but run at **lowest priority** and may be **preempted** (killed) when paying-tier jobs need GPUs.

- **Max wall time:** 7 days
- **Max GPUs per job:** 36 GPUs (9 nodes x 4 GPUs)
- **SU cost:** FREE - no SUs deducted (your budget is already exhausted)
- **Scheduling:** Lowest priority - fills idle GPUs via backfill only
- **Preemption:** Your job may be killed to make room for priority/standard/long jobs
- **Use for:** Work that can tolerate interruptions and has checkpointing
- **WARNING: Always checkpoint:** Burst jobs can be killed at any time. `--requeue --signal=B:SIGTERM@900` is essential

Running jobs are **not** killed when your budget runs out - they finish normally. Only new submissions are blocked until the system moves you to burst.

■ For the full comparison table with priority values, preemption rules, and flags -> see the **QoS Policy** tab.

SU Billing

SU = GPUs x Hours x SU Rate - Example: 4 GPUs x 10h on standard = 4 x 10 x 1.0 = **40 SU**

Real-World Scenarios

Based on Empire AI project workloads - here's how to plan your jobs:

LLM Fine-tuning (7B-40B params)

standard or long

4-16 GPUs (1-4 nodes), 1-5 days. Exceeds 48h wall? Use **long** (7d max, half cost). **Always** add `--requeue`
`--signal=B:SIGTERM@900` + checkpointing.

Foundation Model Training (multi-node)

standard + requeue

16 GPUs across 4 nodes, days-weeks. Use `--nodes=4` `--gres=gpu:4`. Checkpoint every epoch so a requeued job can resume instead of starting over.

Quick Script Test / Debug

test or interactive

Verify your code runs before committing SUs. Test = batch (half price, 6h). Interactive = live shell (2h). Always test first.

New User - First Day on the Cluster

interactive -> test -> standard

Start with an **interactive** session to verify CUDA, install packages, and explore the environment. Then submit your script on **test** (half price) to confirm it runs without errors. Once verified, run the real job on **standard** with requeue + checkpointing. See the **QoS Policy** tab for details.

QoS Policy

Quick Decision Guide

Click any tier row to expand details. **SU = GPUs x Hours x Tier Rate.**

Tier	Priority	Max Wall	Max GPUs/User	Max Running Jobs	SU Rate	Preempts	Flags
priority	1000	24 hours	72	unlimited	2.0x	burst	DenyOnLimit, OverPartQOS, UsageFactorSafe
test	800	6 hours	4	unlimited	0.5x	burst	DenyOnLimit, OverPartQOS, UsageFactorSafe
interactive	500	2 hours	4	1	1.0x	burst	DenyOnLimit, OverPartQOS
standard	500	2 days	36	unlimited	1.0x	burst	DenyOnLimit, OverPartQOS
long	200	7 days	36	unlimited	0.5x	burst	DenyOnLimit, NoReserve, OverPartQOS, UsageFactorSafe
burst	0	7 days	36	unlimited	FREE	-	NoReserve

priority - 2x cost, highest scheduling priority. Use for deadline-critical work only. Up to 72 GPUs (18 nodes). **DenyOnLimit** rejects (not queues) jobs exceeding limits.

test - Half-price sandbox. Capped at 4 GPUs / 6h to prevent accidental burns. High priority (800) so tests start fast. Use before every real run.

interactive - Live GPU shell via `srunk --qos=interactive --pty bash`. 4 GPUs, 2h, 1 session only. Session dies on SSH disconnect (use `tmux`).

standard - Default production tier (used if `--qos` is omitted). Up to 36 GPUs, 48h. Scheduler reserves resources for you. Most workloads belong here.

long - Half-price (0.5x), 7-day wall time, but lower priority (200). **NoReserve** - only fills idle gaps. Great for overnight/weekend runs.

burst - Free, system-assigned when your project runs out of SUs. Cannot be selected manually. Preemptable by all other tiers. Always use checkpointing.

Preemption Chain

Higher-priority tiers can preempt (kill and requeue) burst jobs. No other tier is preemptable.

WARNING: Requeue != Instant Restart

Requeued jobs go back to **pending** and compete with all other jobs. Your fairshare drops after consuming GPU-hours, so others may start first. **Always checkpoint** so a delayed restart doesn't lose progress. Check your fairshare: `sshare -u $USER -l`

Three-Layer Enforcement

Every job passes through **three independent limit checks**. If any layer blocks, the job is rejected:

Layer	Scope	What It Controls	Example
1. QoS (per-user)	Your chosen tier	Max GPUs per user, wall time, SU rate, preemption	standard: max 36 GPUs per job, 48h
2. Account (per-project)	Your project's allocation	Total SU budget (GrpTRESMins), managed by your PI	Project has 10,000 SU - when exhausted, new jobs blocked
3. Institution QoS	Your university	Max concurrent GPUs across all users at your institution	Institution-level limits managed via ColdFront allocations

■ Budget Exhaustion - What Happens?

Running jobs are NOT killed when your project's SU budget runs out - they finish normally. Only **new submissions** are blocked. The system then moves your project to the **burst** tier automatically, where you can keep working for free at lowest priority.

Reference & FAQ

Core Concepts

Service Unit (SU) The currency of the cluster - 1 SU = 1 GPU-hour at standard rate

Every job costs SUs from your project's budget. Formula: **SU = GPUs x Hours x Base Rate x SU Multiplier**

QoS	Multiplier	4 GPUs x 10h
priority	2.0x	80 SU
standard	1.0x	40 SU
test / long	0.5x	20 SU
burst	FREE	0 SU

Check balance with `sshare -u $USER -l`. Billing is automatic - no action needed.

Quality of Service (QoS) Your service level - controls priority, cost, and limits

Set with `--qos=TIER`. Determines how fast your job starts, how much it costs, and resource limits. Five user-selectable tiers: priority, test, interactive, standard, long. A sixth tier (burst) is system-assigned only.

Partition Hardware pool - which GPUs your job runs on

Beta has a single shared partition: beta. All users submit to the same partition. Use `--partition=beta` in all job submissions.

Hardware: 72 NVIDIA GB200 NVL72 nodes, 4 GPUs per node (288 GPUs total), 189 GB HBM3e per GPU, NVLink interconnect.

To see your partition: `sacctmgr show assoc user=$USER format=Partition%15`

Wall Time Max clock time for your job - exceeded = killed

Set with `--time=HH:MM:SS` or `--time=D-HH:MM:SS`. If your job exceeds the wall time, Slurm terminates it. Shorter wall times help backfill scheduling - your job may start sooner.

Fairshare Balances access - heavy recent usage lowers your priority

Slurm tracks how much your project has used recently. The more you've used, the lower your scheduling priority for new jobs. Effect fades over ~14 days. Works at the institution level first, then per-user within the project.

Allocation Your project's approved SU budget

An approved SU grant for a period (e.g., 700K SUs/quarter). Managed by your PI. When exhausted, jobs may be held or moved to burst.

Account Your Slurm project (e.g., columbia_lab1)

Maps to a project in the allocation system. If you belong to multiple projects, specify with `--account=PROJECT`. Your SU budget is shared among all members of the account.

Commands

sbatch Submit a batch job script - runs in the background

Submits a job script to Slurm. Returns a job ID. The job runs unattended on compute nodes. Use for training, inference, and any work that doesn't need a live terminal.

```
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=standard --gres=gpu:4 --time=12:00:00 train.sh
```

srslun Run a command interactively on a compute node

Launches a command directly on a compute node. Use `--pty bash` for a live shell. The session ends when you disconnect or hit the time limit.

```
srslun --partition=beta --account=YOUR_ACCOUNT --qos=interactive --gres=gpu:1 --time=01:00:00 --pty bash
```

squeue Check your running and pending jobs

```
# Your jobs
squeue -u $USER

# All jobs on the cluster
squeue
```

scancel Cancel a running or pending job

```
# Cancel one job
scancel <jobid>

# Cancel all your jobs
scancel -u $USER
```

sacct View completed job details and SU usage

```
sacct -u $USER --format=JobID%10,JobName%20,QOS%12,AllocTRES%35,Elapsed,State -S today
```

sinfo Check cluster node availability and partition status

```
# Summary
sinfo -s

# Detailed node list
sinfo -N -l
```

Job Types & Features

Job Array Submit many independent tasks with one command

Submit N copies of the same script with `--array=1-N`. Each task gets a unique `$SLURM_ARRAY_TASK_ID` your script uses to select parameters, configs, or data shards. Tasks schedule independently - faster than multi-node since they fill gaps individually.

Use for: hyperparameter sweeps, batch inference, data preprocessing, ablation studies.

```
sbatch --partition=beta --account=YOUR_ACCOUNT --qos=standard --gres=gpu:1 --time=4:00:00 --array=1-100
sweep.sh
```

Output: `slurm-JOBID_1.out`, `slurm-JOBID_2.out`, etc. See **Setup, Test & Verify** tab for a test example.

Interactive Session Live terminal on a GPU node for debugging and exploration

A live shell on a compute node. Limited to 1 session, 4 GPUs, 2 hours. Runs via `srun` (not `sbatch`). Use for debugging with `gdb/pdb`, data exploration, environment setup, quick prototyping.

Requeue Make a job eligible to restart after preemption, node failure, or requeue

`--requeue` makes the job eligible to go back into the queue after a requeue event. Each restarted run is billed separately. **Always combine with checkpointing** - without it, the job starts from scratch. See the Setup, Test & Verify tab for examples.

Checkpointing Save training state to disk - resume after restart (NVIDIA best practice)

Periodically save model weights, optimizer state, and epoch to disk. If interrupted (wall time, preemption, node failure), resume from the last checkpoint instead of starting over.

Key pattern: `--requeue --signal=B:SIGTERM@900` -> Slurm warns your script 15 min before wall time -> script saves checkpoint and exits -> job requeues -> resumes from checkpoint on restart.

The recommended template keeps only the **last 3 checkpoints** and auto-deletes older ones. Store in `$HOME/checkpoints/`.

Without checkpointing, a requeued job starts from scratch - losing all progress.

See the **Setup, Test & Verify** tab for complete PyTorch code.

Epoch One complete pass through your entire training dataset

One full cycle where the model processes every sample in your training data. Training runs for many epochs (10-100+). Checkpoints are typically saved after each epoch so interrupted jobs resume from the last completed one.

Multi-node Job Run across multiple servers for large-scale training

Use `--nodes=N` to request multiple servers. Each Beta node has 4 GPUs. Example: `--nodes=4 --gres=gpu:4` gives you 16 GPUs across 4 nodes. Requires distributed training code (e.g., PyTorch DDP, DeepSpeed).

Containers (Pyxis + Enroot) Run GPU workloads inside NGC containers - the primary way to use PyTorch on Beta

Beta compute nodes have **no pre-installed Python or PyTorch**. All ML workloads run inside NVIDIA NGC containers via the Pyxis/Enroot runtime. Use `--container-image=nvcr.io/nvidia/pytorch:26.02-py3` in your sbatch or srun command.

Key flags:

- `--container-image=` - NGC container to pull and run
- `--container-name=` - cache the container locally (faster restarts)
- `--container-mounts=` - bind-mount host directories into the container
- `--container-mount-home` - mount your home directory inside the container

Note on sm_100: PyTorch 26.02 detects all 4 GB200 GPUs but displays a `sm_100 is not compatible` warning - basic operations work, but native Blackwell CUDA kernels are not yet included. This does not prevent container usage or GPU detection. A future NGC release will add full sm_100 support. Do **not** use 24.12-py3 or earlier - those lack GB200 support entirely.

Resources & Limits

GRES (Generic Resources) How Slurm tracks GPUs - use `gpu:N` or `gpu:type:N`

Use `--gres=gpu:N` for any available GPU, or specify a type explicitly.

GPU	VRAM	Per Node	Request
NVIDIA GB200 (NVL72)	189 GB HBM3e	4	<code>--gres=gpu:N (N <= 4)</code>

72 NVIDIA GB200 NVL72 nodes, 4 GPUs per node (288 GPUs total), 189 GB HBM3e per GPU. All nodes have the same GPU type - no need to specify GPU model.

Node Single physical server - 4 GB200 GPUs + CPUs + memory

Each Beta node has 4 NVIDIA GB200 GPUs (189 GB HBM3e each). Multi-node jobs span multiple servers via NVLink interconnect. The test QoS is limited to 4 GPUs (1 node) max.

DenyOnLimit Jobs exceeding QoS limits rejected immediately with a clear error

Instead of silently queuing a job that can never run, Slurm rejects it at submission with a descriptive error. All tiers except burst have this flag enabled.

UsageFactorSafe Slurm checks your weighted SU cost against your budget before accepting the job

Applied to tiers with non-standard rates (priority 2.0x, test 0.5x, long 0.5x). Ensures Slurm compares the **actual cost** (GPU-hours x tier rate) against your remaining budget at submit time - preventing jobs from being accepted then killed mid-run when the weighted cost exceeds your balance.

NoReserve Long/burst jobs only fill idle gaps - may wait longer to start

Applied to long and burst tiers. The scheduler won't hold GPUs for these jobs - they only start when naturally idle resources appear. This means they may wait longer, but it keeps cheap/free jobs from blocking higher-priority work. **Tip:** Request shorter wall times to improve backfill chances.

Preemption Higher-priority jobs can cancel running burst jobs

All paid tiers (priority, test, interactive, standard, long) can preempt burst jobs. If you're on burst and a paid job needs your GPUs, your job gets terminated. This is why checkpointing is essential for burst/long.

Backfill Slurm fills scheduling gaps with smaller jobs that fit

If there's a gap before the next large job starts, Slurm fills it with shorter jobs that will finish in time. Shorter wall times and fewer GPUs = more backfill opportunities = faster start.

Default QoS Behavior Jobs submitted without --qos= automatically use standard

If you omit --qos= from your submission, your job automatically runs on the standard tier (1.0x rate, 48h max, 36 GPUs/job max). This is Slurm's native DefaultQOS behavior - no special plugin required. To use a different tier (test, long, priority, interactive), you must explicitly specify it with --qos=TIER.

Frequently Asked Questions

Common questions based on real Empire AI project workloads. Click any question to expand the answer.

My job is stuck in PENDING - why isn't it starting?

Common reasons: **(1)** Not enough free GPUs - other jobs are using them. **(2)** Your fairshare is low from heavy recent usage (priority drops after heavy use, recovers over ~14 days). **(3)** You requested more resources than available (e.g., 36 GPUs when only 16 are free). **Tip:** Use `squeue -u $USER` to check status and `sinfo -s` to see cluster availability. Request fewer GPUs or shorter wall time to start sooner via backfill.

My job was killed before it finished - what happened?

(1) Wall time exceeded: Your job hit the tier's max time (e.g., 48h for standard). Fix: add `--requeue --signal=B:SIGTERM@900` + checkpointing. **(2) Out of memory:** Your job used more RAM than requested. Fix: increase `--mem`. **(3) Preemption:** Only affects burst tier - a higher-priority job needed your GPUs. Fix: use a paid tier. Check `sacct -u $USER -S today` for the "State" column (TIMEOUT, OUT_OF_MEMORY, PREEMPTED).

How do I run a training job that takes weeks?

No tier allows unlimited runtime (max is 7 days on long). Use **requeue + checkpointing**: add `--requeue --signal=B:SIGTERM@900` to your sbatch command and save model checkpoints in your training script. If the job is requeued, it can resume from the last checkpoint instead of starting from scratch. See the **Setup, Test & Verify** tab for complete PyTorch code. This is the standard approach for all multi-day training - LLM fine-tuning, foundation models, molecular dynamics, etc.

How many SUs will my job cost?

Formula: **SU = GPUs x Hours x Tier Multiplier**. Example: 4 GPUs for 24 hours on standard (1.0x) = **96 SU**. Same job on long (0.5x) = **48 SU**. On priority (2.0x) = **192 SU**. Check your balance with `sshare -u $USER -l`. If your project runs out of SUs, jobs auto-move to burst (free but lowest priority).

I got an error about GPU type - what do I put?

Use `--gres=gpu:N` for any available GPU. If you need a specific type, use the full GRES name. On Beta, all 72 nodes have identical GB200 GPUs - just use `--gres=gpu:N` ($N \leq 4$). An error means the GPU count exceeds per-node capacity or the type name is invalid.

Where do I store my scripts and data?

Everything goes in your home directory: `/mnt/home/$USER/` (15 PB shared storage). Store scripts in `~/scripts/`, data in `~/data/`, and checkpoints are auto-managed in `~/checkpoints/` by the recommended PyTorch template (see Setup, Test & Verify tab). Job output files land in the directory you submit from - use `--output` and `--error` directives to control log locations.

Does Beta have different GPU types?

Beta has a single GPU type: NVIDIA GB200 with 189 GB HBM3e per GPU, 4 GPUs per node. Simply use `--gres=gpu:N` ($N \leq 4$). No need to specify GPU model - all 72 nodes have the same hardware.

How do I run a multi-node distributed training job?

Use `--nodes=N --gres=gpu:4` (4 GPUs per node). Example: 4 nodes = 16 GPUs. Your training code must support distributed training (PyTorch DDP, DeepSpeed, FSDP). Max GPUs per tier: standard = 36 GPUs/job, long = 36 GPUs/job, priority = 72 GPUs/job. Always use requeue + checkpointing for multi-node jobs.

How do I check my SU balance and usage?

Use these command-line tools to check your balance and usage: `sshare -u $USER -l` shows your fairshare usage. `sacct -u $USER --format=JobID,JobName,QoS,AllocTRES,Elapsed -S 2026-01-01` shows all your jobs and resources used. (ColdFront portal at `coldfront.empireai.edu` is planned but not yet in production.)

Will my requested job restart immediately?

Not guaranteed. When your job hits the wall time and requeues, it goes back into the pending queue as a new submission. The scheduler re-evaluates using **fairshare** - your usage just increased, so your priority drops. If another user with better fairshare has a pending job, theirs goes first and yours stays PENDING. Fairshare recovers over ~14 days. **Tips:** request shorter wall times (more backfill-friendly), use fewer GPUs, and checkpoint frequently so PENDING time doesn't lose progress. See the **QoS Policy** tab for the full step-by-step breakdown.

Why would I use test instead of long if they're both 0.5x SU?

They're the same price per GPU-hour, but designed for different jobs:

test is a sandbox - capped at 6 hours and 4 GPUs. It schedules fast (priority 800) because it's small. The caps prevent you from accidentally burning a week of GPU time on a "quick test." Use it for: verifying scripts work, testing data loading, checking multi-GPU code.

long is budget production - up to 7 days and 36 GPUs per job. It schedules slower (priority 200, NoReserve) because it's competing for real resources. Use it for: long training runs, overnight/weekend jobs, any real workload where you want to save SUs.

Rule of thumb: If your job fits in 6 hours on 4 GPUs, use test. If it's longer or bigger, use long. See the **QoS Policy** tab for details.

What's the difference between interactive and test?

The difference is **how you interact with the job**:

interactive (`srun --pty bash`) gives you a live terminal on a GPU node - like SSH. You type commands in real-time. Use it for debugging, installing packages, data exploration. Limited to 4 GPUs, 2 hours, 1 session at a time. Costs 1.0x SU. Session dies if your SSH disconnects.

test (`sbatch`) runs a script in the background - you get results in a log file. Use it for verifying your training script runs without errors before committing to a full run. Up to 4 GPUs per job, 6 hours, no batch job-count cap. Costs 0.5x SU. Survives SSH disconnects.

Typical workflow: Start with interactive to set up your environment and prototype. Then switch to test to verify your script works as a batch job. Then submit the real run on standard or long. See the **QoS Policy** tab for details.

What actually happens when my project runs out of SUs? Will my running jobs be killed?

Running jobs are NOT killed. They finish normally. Only **new submissions** are affected.

When your project's SU budget is exhausted, `budget_enforcer.sh` automatically moves your project to the **burst** tier. You can still submit jobs - they're free (0 SU cost) - but they run at the lowest priority (0) and are **preemptable** by any paid-tier job.

What preemptable means in practice: On a busy cluster, burst jobs may get killed repeatedly and never make meaningful progress. As Slurm consultant Joshi Fullop noted: *"The preemptable jobs pretty much don't get run or don't make any appreciable headway. It's the same thing as just cutting them off on a fully subscribed system."*

To restore normal access: Your PI can request additional SUs through your institution's allocation process. Until then, always use `--requeue --signal=B:SIGTERM@900` + checkpointing on burst jobs to survive preemption.

Common Error Messages

Sorted alphabetically. Titles are the **exact error text** you'll see in your terminal or `sacct` output. Errors marked were verified on this cluster.

Access denied by pam_slurm_adapt: you have no active jobs on this node

Cause: You tried to SSH directly to a compute node without a running job on it.

Fix: Use `srun --partition=beta --account=YOUR_ACCOUNT --qos=interactive --gres=gpu:1 --time=01:00:00 --pty bash` to get a shell, or submit with `sbatch`. You cannot SSH to compute nodes directly.

CUDA out of memory / RuntimeError: CUDA error

Cause: Your model or batch size exceeds GPU VRAM. GB200 = 189 GB HBM3e. This is a PyTorch error, not Slurm - your job started but crashed.

Fix: Reduce batch size, use mixed precision (fp16/bf16), enable gradient checkpointing, or use multi-node to distribute across more GPUs: `--nodes=2 --gres=gpu:4`.

ModuleNotFoundError: No module named 'torch'

Cause: No system-wide Python or PyTorch. Your script used the default system Python which has no ML libraries.

Fix: Use an NGC container via `--container-image=nvcr.io/nvidia/pytorch:26.02-py3` in your sbatch command (or add `#SBATCH --container-image=...` to your script). See **Section 1** on the Setup, Test & Verify tab.

NVIDIA GB200 (sm_100) is not supported by the current PyTorch / CUDA container

May appear as: `UserWarning: ...sm_100 is not compatible..., CUDA error: no kernel image is available for execution on the device, or training simply produces NaN/garbage results.`

Cause: Your NGC container is too old. GB200 uses the `sm_100` compute capability (Blackwell). Containers built before early 2025 don't include `sm_100` kernels.

Fix: Use `--container-image=nvcr.io/nvidia/pytorch:26.02-py3` or newer. PyTorch 26.02 detects all 4 GB200 GPUs correctly but shows a `sm_100 is not compatible` warning - this is expected and does not block GPU detection or basic operations. Native Blackwell CUDA kernels will be included in a future NGC release. Do **not** use 24.12-py3 or earlier - those lack GB200 support entirely.

module: not found

Full error: `/cm/local/apps/slurm/var/spool/jobNNNNNN/slurm_script: N: module: not found`

Cause: The `module` command (used on other HPC clusters) does not exist on Empire AI.

Fix: Remove any `module load` lines from your scripts. Use NGC containers via `--container-image` instead. CUDA and GPU drivers are pre-configured on Beta compute nodes.

sbatch: error: Batch job submission failed: Invalid partition name specified

Full error (two lines):

```
sbatch: error: invalid partition specified: alpha
sbatch: error: Batch job submission failed: Invalid partition name specified
```

Cause: Partition name is misspelled or missing. Beta uses a single partition: `beta`.

Fix: Add `--partition=beta` to your submission. Verify your association: `sacctmgr show assoc user=$USER format=Partition%15,Account%25`

sbatch: error: Batch job submission failed: Invalid qos specification

Cause: The `--qos=` tier you requested is not assigned to your account, or doesn't exist.

Fix: Check your tiers: `sacctmgr show assoc where user=$USER format=Account%20,QOS%60`. Valid tiers: `test`, `interactive`, `standard`, `long`, `priority`. Note: `burst` is auto-assigned, not user-selectable.

sbatch: error: Batch job submission failed: Job violates accounting/QOS policy (job submit limit, user's size and/or time limits)

This error appears as a second line after a specific reason like `QOSMaxGRESPerUser`, `QOSMaxWallDurationPerJobLimit`, or `QOSMaxJobsPerUserLimit`. Common triggers:

- **Too many GPUs** - e.g., `--gres=gpu:8` on `test` (max 4)
- **Wall time too long** - e.g., `--time=12:00:00` on `test` (max 6h)
- **Too many running jobs** - e.g., 2nd `interactive` session (max 1)

Fix: Reduce your request to fit the tier limits (see **QoS Tiers** tab), or switch to a tier with higher limits.

sbatch: error: Batch job submission failed: Requested node configuration is not available

Cause: You requested more GPUs than a single node has (each Beta node = 4 GPUs), or the GPU type name is invalid.

Fix: For >4 GPUs use multi-node: `--nodes=2 --gres=gpu:4` (= 8 GPUs). Beta has a single GPU type (GB200) - just use `--gres=gpu:N` where `N <= 4`.

sbatch: error: invalid account or account/partition combination specified

Cause: The `--account=` name doesn't exist, or your account isn't authorized for the partition. Also happens if you have multiple projects and don't specify which one.

Fix: Check your accounts: `sacctmgr show assoc where user=$USER format=Account%25,Partition%20,QOS%60`. If you have multiple accounts, add `--account=YOUR_ACCOUNT` to your command.

sbatch: error: QOSMaxGRESPerUser

Full error (two lines):

```
sbatch: error: QOSMaxGRESPerUser
sbatch: error: Batch job submission failed: Job violates accounting/QoS policy (job submit limit, user's
size and/or time limits)
```

Cause: Your job requests more GPUs than the per-job GPU limit for that tier.

Fix: Reduce `--gres=gpu:N`, wait for current jobs to finish, or cancel unneeded jobs: `scancel <jobid>`. Check running jobs: `squeue -u $USER`.

sbatch: error: QOSMaxGRESPerJob

Full error (two lines):

```
sbatch: error: QOSMaxGRESPerJob
sbatch: error: Batch job submission failed: Job violates accounting/QoS policy (job submit limit, user's
size and/or time limits)
```

Cause: Your **single job** requests more GPUs than the per-job limit for that tier. Example: `--gres=gpu:8` on test (max 4 GPUs per job).

vs QOSMaxGRESPerUser: PerJob = one job exceeds the limit. PerUser = your **combined** running jobs exceed the limit.

Fix: Reduce `--gres=gpu:N`, or switch to a tier with higher limits (standard allows 36 GPUs/job).

sbatch: error: QOSMaxWallDurationPerJobLimit

Full error (two lines):

```
sbatch: error: QOSMaxWallDurationPerJobLimit
sbatch: error: Batch job submission failed: Job violates accounting/QoS policy (job submit limit, user's
size and/or time limits)
```

Cause: Your requested `--time=` exceeds the maximum wall time for that QoS tier.

Fix: Reduce `--time=`, or switch to a tier with a longer wall-time limit. For long-running work, use `--requeue --signal=B:SIGTERM@900` and checkpointing.

sbatch: error: QOSMaxJobsPerUserLimit

Cause: You've hit the max running jobs for this tier. The interactive tier allows only **1 job at a time**.

Fix: Wait for your current session to end, or cancel it: `scancel <jobid>`. For batch work, use standard or test - they have no batch job-count cap.

slurmstepd: error: *** JOB ... CANCELLED AT ... DUE TO TIME LIMIT ***

Actual output:

```
slurmstepd: error: *** JOB 12345 ON b1-12-s1-dgx-01-c02 CANCELLED AT 2026-06-10T12:00:00 DUE TO TIME
LIMIT ***
```

Cause: Your job hit its wall time limit. `sacct` will show state = **TIMEOUT**. This is normal for long training runs.

Fix: Add `--requeue --signal=B:SIGTERM@900` and implement checkpointing. Your job will save state 15 min before the limit, then auto-restart. See **Requeue & Checkpointing** on the Setup, Test & Verify tab.

srun: job XXXXX queued and waiting for resources

Cause: Your `srun` interactive session is waiting for a GPU to become available. Unlike `sbatch` which returns immediately, `srun` blocks until resources are allocated.

Fix: Wait - your session will start when a GPU frees up. If it takes too long, press **Ctrl+C** to cancel and try again with fewer GPUs or a shorter time. Check cluster load: `squeue -u $USER`.

squeue shows PENDING with Reason=Priority or Reason=Resources

Cause: The cluster is busy. **Priority** = other jobs have higher scheduling priority. **Resources** = not enough immediately usable GPUs/nodes free right now. A node can show idle CPUs or configured GPUs but still be unavailable if it is draining, completing, or reserved by the scheduler for higher-priority/backfill work.

Fix: This is normal - your job starts when resources free up. To start faster: request fewer GPUs, shorter wall time (helps backfill), or use `--qos=priority` (2x SU cost). Check queue: `sprio -j <jobid>`.

squeue shows PENDING with Reason=PartitionConfig

Cause: Slurm accepted the job but the selected partition does not allow the account or partition association Slurm chose for your user. This can happen if you have multiple accounts and your default account is not allowed on the partition you requested.

Fix: Verify your account/partition mapping: `sacctmgr show assoc where user=$USER format=Account%25,Partition%20,QOS%60`. Beta has a single partition (beta). Make sure you include `--account=YOUR_ACCOUNT` with the correct account name from your association list.

State: PREEMPTED in sacct output

Cause: Your burst tier job was killed because a paid-tier job needed the GPUs. Only burst jobs get preempted - paid tiers are never preempted.

Fix: Expected for burst. Always use `--requeue --signal=B:SIGTERM@900` + checkpointing for burst jobs. If you need guaranteed runtime, use a paid tier.

Need Help?

Submit a ticket via **Freshdesk** or email your institution's technical point of contact. Include your job ID (`squeue -u $USER`) and any error messages.